

5.3 参照完整性

5.3.1 定义参照完整性

关系模型的参照完整性在 CREATE TABLE 中用 FOREIGN KEY 短语定义哪些列为外码, 用 REFERENCES 短语指明这些外码参照哪些表的主码。

例如, 关系 SC 中一个元组表示一个学生选修某门课程的成绩, (Sno, Cno) 是主码。Sno、Cno 分别参照引用 Student 表的主码和 Course 表的主码。

[例 5.3] 定义 SC 中的参照完整性。

```
CREATE TABLE SC
(Sno CHAR(8),
 Cno CHAR(5),
 Grade SMALLINT,
 Semester CHAR(5),
 Teachingclass CHAR(8),
 PRIMARY KEY(Sno, Cno),      /* 在表级定义实体完整性 */
 FOREIGN KEY(Sno) REFERENCES Student(Sno),
                               /* 在表级定义参照完整性, Sno 是外码, 被参照表是 Student */
 FOREIGN KEY(Cno) REFERENCES Course(Cno)
                               /* 在表级定义参照完整性, Cno 是外码, 被参照表是 Course */
);
```

5.3.2 参照完整性检查和违约处理

参照完整性将两个表中的相应元组联系起来了。因此, 对被参照表和参照表进行更新(增、删、改)操作时有可能破坏参照完整性, 必须进行检查以保证这两个表的相容性。

例如, 对表 SC 和 Student 有 4 种可能破坏参照完整性的情况及违约处理, 如表 5.1 所示。

表 5.1 可能破坏参照完整性的情况及违约处理

被参照表 (如 Student)	参照表 (如 SC)	违约处理
可能破坏参照完整性	插入元组	拒绝
可能破坏参照完整性	修改外码值	拒绝
删除元组	可能破坏参照完整性	拒绝/级联删除/设置为空值
修改主码值	可能破坏参照完整性	拒绝/级联修改/设置为空值

① SC 表中增加一个元组, 该元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

② 修改 SC 表中的一个元组, 修改后该元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

③ 从 Student 表中删除一个元组, 造成 SC 表中某些元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

④ 修改 Student 表中一个元组的 Sno 属性, 造成 SC 表中某些元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

当上述不一致发生时, 系统可以采用以下策略加以处理。

① 拒绝执行。不允许该操作执行。该策略一般设置为默认策略。

② 级联操作。当删除或修改被参照表(Student)的一个元组导致与参照表(SC)的不一致时, 删除或修改参照表中所有导致不一致的元组。

例如, 删除 Student 表中 Sno 值为“20180001”的元组, 则要从 SC 表中级联删除“SC. Sno = '20180001'”的所有元组。

③ 设置为空值。当删除或修改被参照表的一个元组时造成不一致, 则将参照表中所有造成不一致的元组的对应属性设置为空值。例如, 下面两个关系:

学生(学号, 姓名, 性别, 出生日期, 主修专业)

专业(专业名, 专业编码)

其中“学生”关系的“主修专业”是外码, 参照“专业”关系的主码“专业名”。

假设“专业”表中“专业名”为计算机科学与技术的元组被删除, 按照表 5.1 的策略, 就要把“学生”表中“主修专业 = ‘计算机科学与技术’”的所有元组的主修专业设置为空值。这样处理的语义可以解释为: 某个专业删除了, 则选择该专业为主修专业的所有学生就要等待重新选择主修专业。

这里讲解一下外码能否接受空值的问题。

例如, “学生”表中“主修专业”是外码, 按照应用的实际情况可以取空值, 表示这个学生的主修专业尚未确定。

但在“学生选课”数据库中, 关系 SC 的 Sno 和 Cno 为外码, 它能否取空值呢? 答案是“不能”。因为 Sno 和 Cno 是关系 SC 的主属性, 按照实体完整性约束条件, 主属性不能为空值。若 SC 的 Sno 为空值, 则表明尚不存在的某个学生或者某个不知学号的学生选修了某门课程, 其成绩记录在 Grade 列中。这与学校的应用环境是不相符的, 因此 SC 的 Sno 列不能取空值。同样, SC 的 Cno 列也不能取空值。

因此对于参照完整性, 除了应该定义外码, 还应定义外码列是否允许空值。

一般地, 当对参照表和被参照表的操作违反了参照完整性时, 系统选用默认策略, 即拒绝执行。如果想让系统采用其他策略, 则必须在创建参照表时显式地加以说明。

[例 5.4] 显式说明参照完整性的违约处理示例。

```
CREATE TABLE SC
(
    Sno CHAR(8),
    Cno CHAR(5),
    Grade SMALLINT,      /* 成绩 */
    Semester CHAR(5),    /* 开课学期 */
    Teachingclass CHAR(8), /* 学生选修某一门课所在的教学班 */
    PRIMARY KEY(Sno, Cno), /* 在表级定义实体完整性, Sno、Cno 都不能取空值 */
    FOREIGN KEY(Sno) REFERENCES Student(Sno) /* 在表级定义参照完整性 */
    ON DELETE CASCADE /* 当删除 Student 表中的元组时, */
                        /* 级联删除 SC 表中 Sno 与 Student.Sno 相同的 SC 元组 */
    ON UPDATE CASCADE, /* 当更新 Student 表中的 Sno 时, */
                        /* 级联更新 SC 表中相应元组的 Sno */
    FOREIGN KEY(Cno) REFERENCES Course(Cno)
                        /* 在表级定义参照完整性 */
    ON DELETE NO ACTION /* 当删除 Course 表中的元组造成与 SC 表不一致, 即 SC 中存在
                        /* 这样的元组, 其 Cno 与 Course.Cno 相同, 则拒绝删除 Course
                        /* 表的这个元组 */
    ON UPDATE CASCADE /* 当更新 Course 表中的 Cno 时, */
                        /* 级联更新 SC 表中相应元组的 Cno */
);
```

可以对 DELETE 和 UPDATE 采用不同的策略。例如, 例 5.4 中当删除被参照表 Course 表中的元组, 造成与参照表(SC 表)不一致时, 拒绝删除被参照表的元组; 对更新操作则采用级联更新的策略。

从上面的讨论可以看到, 关系数据库管理系统在实现参照完整性时, 除了要提供定义主码、外码的机制外, 还需要提供不同的策略供用户选择。具体选择哪种策略, 要根据应用环境的要求确定。

5.4 用户定义的完整性

用户定义的完整性就是某一具体应用涉及的数据必须满足的语义要求。目前的关系数据库管理系统都提供了定义和检查这类完整性的机制, 使用和实体完整性、参照完整性相同的技术和方法来处理它们, 而不必由应用程序实现这一功能。